AZURE DATABASE ADMINISTRATOR

200 Real-time FAQs — Volume 2

Advanced scenario-based interview & operations reference

Drafted by	Praveen Madupu
Handle	PMSQLDBA | sqldbachamps.com
Date	7th July, 2026 - Tuesday
Document	Azure DBA - 200 Real-time FAQs (Volume 2)
Format	Point-wise answers on real-time scenarios
Version	2.0

13 domains • 200 fresh non-overlapping questions • advanced production scenarios

Table of Contents

Table of Contents 2
How to Use This Reference 3
1. Azure SQL Database — Advanced Configuration & Operations 4
2. Azure SQL Managed Instance — Advanced Administration 8
3. SQL Server on Azure Virtual Machines (IaaS) — Advanced 11
4. High Availability & Disaster Recovery — Advanced 14
5. Backup, Restore & Recovery 18
6. Performance Tuning & Query Optimization 21
7. Security, Compliance & Data Protection 25
8. Monitoring, Alerting & Diagnostics 29
9. Migration & Modernization 32
10. Automation, DevOps, IaC & Job Orchestration 35
11. Elastic Pools, Serverless & Hyperscale 38
12. Networking & Connectivity 40
13. Cost Optimization, Licensing & Governance 41
End of Document 41

How to Use This Reference

This reference contains 200 additional real-time, scenario-based questions and point-wise answers spanning the full remit of an Azure Database Administrator across PaaS (Azure SQL Database, Managed Instance) and IaaS (SQL Server on Azure VM). This is Volume 2 and does not repeat any question from Volume 1 (the 300-FAQ set).

Questions are grouped into 13 domains. Each answer is written as concise, actionable bullet points reflecting how the situation is handled in production - not textbook definitions.

Use it for interview preparation, on-call runbook thinking, and validating your own operational approach. Where a decision is involved, the answer explains the reasoning, not just the button to press.

> **Reading tip**
> Answers assume a real environment: least-privilege security, HA/DR by design, cost-awareness, and validation before cutover. Adapt thresholds and tiers to your workload.

Domain Summary

#	Domain	Questions
1	Azure SQL Database — Advanced Configuration & Operations	18
2	Azure SQL Managed Instance — Advanced Administration	18
3	SQL Server on Azure Virtual Machines (IaaS) — Advanced	15
4	High Availability & Disaster Recovery — Advanced	22
5	Backup, Restore & Recovery	16
6	Performance Tuning & Query Optimization	22
7	Security, Compliance & Data Protection	20
8	Monitoring, Alerting & Diagnostics	16
9	Migration & Modernization	16
10	Automation, DevOps, IaC & Job Orchestration	14
11	Elastic Pools, Serverless & Hyperscale	12
12	Networking & Connectivity	6
13	Cost Optimization, Licensing & Governance	5
TOTAL		**200**

1. Azure SQL Database — Advanced Configuration & Operations

DOMAIN 1

Azure SQL Database — Advanced Configuration & Operations

18 real-time scenarios

Q1. Users report intermittent 'database is not currently available' errors lasting a few seconds. What is happening and how do you make the app resilient?

Answer:

- These are transient faults from planned reconfigurations, failovers, or throttling — expected in a PaaS platform, not an outage.
- Implement retry logic with exponential backoff in the app or use a resilient driver (EF Core execution strategy, SqlClient retry provider).
- Catch the known transient error numbers (40613, 40197, 40501, 49918) and retry rather than surfacing the failure.
- Confirm connection strings use the FQDN and rely on Gateway redirection; keep connection pooling enabled to reduce reconnect cost.

Q2. A Business Critical database must survive a full Azure region loss with near-zero data loss. What do you configure and what are the trade-offs?

Answer:

- Use an auto-failover group across paired regions for grouped failover and a stable read-write listener endpoint.
- Replication is asynchronous, so a hard region loss can lose the last few seconds of committed transactions — RPO is not truly zero.
- Set the grace period high enough to avoid failing over on brief blips but low enough to meet RTO.
- Size the secondary identically so it can carry full production load after failover.

Q3. The 'Used space' on your Azure SQL Database keeps growing but you deleted a large amount of data. Why, and what do you do?

Answer:

- Deletes free space inside the data file but do not shrink the allocated size; the database keeps the high-water mark.
- Only shrink if the reclaim is large and durable — shrinking causes fragmentation and is usually not worth it.
- If needed, use DBCC SHRINKFILE with a target, then rebuild fragmented indexes afterward.
- For Hyperscale, file management differs — space is handled by the storage layer and manual shrink is not applicable the same way.

Q4. A single database occasionally hits 100% DTU and throttles. Management wants predictability without over-paying. What is your approach?

Answer:

- Review Query Store and the resource-consumption charts to find whether it is CPU, log, or IO-bound.
- Move to the vCore model for transparent, independently sized compute rather than a blended DTU metric.
- Consider Serverless if the spikes are intermittent, or bump one tier if the ceiling is consistently hit.
- Tune the top offending queries first — scaling to mask a missing index wastes money.

Q5. You need to give a reporting team read access to production data without impacting OLTP performance. What is the cleanest option?

Answer:

- Add a read-only replica (available in Business Critical and Hyperscale) and route reports with ApplicationIntent=ReadOnly.
- Reads on the replica do not consume primary compute, isolating the reporting load.

- For heavier isolation, use a geo-replica or a separate copy refreshed on a schedule.
- Grant least-privilege permissions and consider row-level security if they should see only a subset.

Q6. A deployment accidentally changed the database collation expectation and queries now throw collation conflict errors. How do you handle it?

Answer:

- Database collation cannot be changed in place for Azure SQL Database — it is fixed at creation.
- Resolve conflicts per-column or per-expression using COLLATE in the query as an immediate fix.
- For a permanent fix, create a new database with the correct collation and migrate data via BACPAC or data movement.
- Standardize collation in your IaC templates so future deployments are consistent.

Q7. Your app opens and closes thousands of short-lived connections per minute and you see login storms. What do you tune?

Answer:

- Enable and correctly size client-side connection pooling so connections are reused, not recreated.
- Confirm the driver reuses pooled connections with identical connection strings (pools are keyed on the string).
- Reduce per-request open/close churn by scoping connections to the unit of work, not per statement.
- Watch for 'login failed' throttling and back off; consider a connection proxy if the pattern is unavoidable.

Q8. You must ensure a specific database never fails over to a secondary during Microsoft's maintenance without your awareness. What controls exist?

Answer:

- Configure a maintenance window so planned maintenance happens in a predictable off-hours slot.
- Maintenance events still cause a brief reconfiguration — the window controls when, not whether.
- Combine with retry logic so the short reconnect is invisible to users.
- Monitor Resource Health and Service Health alerts to correlate any blips with platform events.

Q9. A developer asks why their new index isn't being used on Azure SQL Database. How do you diagnose it?

Answer:

- Check the actual execution plan — the optimizer may prefer another path due to statistics or estimated cost.
- Verify the index is not disabled and its key/included columns actually cover the query predicates and output.
- Look for implicit conversions or non-SARGable predicates that prevent a seek.
- Update statistics and re-test; consider Query Store to confirm plan choice before and after.

Q10. Finance wants to cap the monthly spend of a non-critical database hard. What options do you present?

Answer:

- Use Serverless with a defined max vCore ceiling and auto-pause to bound compute cost.
- Set a lower service tier and accept the performance ceiling for a non-critical workload.
- Apply an Azure budget with alerts at thresholds so overruns are caught early.
- Right-size storage and disable unneeded features (long-term retention, extra replicas) that add cost.

Q11. Your database periodically shows high LOG_RATE_GOVERNOR waits. What does it mean and what do you do?

Answer:

- The tier's transaction-log throughput ceiling is being hit — writes are being throttled to the tier's log rate.
- Batch and reduce transaction frequency; large single-row-per-commit loops are a classic cause.
- Move bulk operations to minimally logged patterns where supported, or throttle load rate.
- If the workload legitimately needs more log throughput, scale the tier — Business Critical offers higher log rates.

Q12. You need point-in-time visibility of who changed a sensitive table and when, without heavy overhead. What do you enable?

Answer:

- Enable auditing to a Log Analytics workspace or storage account for a durable record of access.
- For data-change history, use temporal tables to keep a queryable row-versioned history.
- For change feeds to downstream systems, use change tracking or change data capture where supported.
- Restrict who can alter audit settings so the trail cannot be silently disabled.

Q13. A database copy operation for a large database is taking hours and blocking a release. How do you plan around it?

Answer:

- Database copy time scales with size and tier — it is a full seeding operation, not instant.
- For very large databases, consider Hyperscale where copy/restore is near-instant due to snapshot-based storage.
- Schedule copies outside the release window and use them as pre-staged test targets.
- For repeatable refreshes, automate the copy and cutover in a runbook rather than doing it by hand.

Q14. Your app needs a globally distributed read footprint with low read latency in three regions. What is the pattern?

Answer:

- Use active geo-replication to create readable secondaries in each target region.
- Route regional read traffic to the nearest secondary using ApplicationIntent or separate endpoints.
- Writes still go to the single primary — accept cross-region write latency for remote users.
- Monitor replication lag so read consumers understand eventual consistency.

Q15. You get an alert that a database is approaching its max size. What is your triage order?

Answer:

- Confirm actual used vs allocated size and the tier's max size cap.
- Reclaim obvious waste — old data, unused indexes, oversized log-side objects — before scaling.
- If growth is legitimate, increase max size within the tier or move to a tier/Hyperscale with more headroom.
- Set an alert threshold below the cap so you are never surprised by a hard stop.

Q16. A team wants to test a risky schema change against production-like data safely. What do you set up?

Answer:

- Create a database copy or restore a point-in-time copy to an isolated database.
- Run the change and workload against the copy; measure duration, blocking, and plan changes.
- Mask or scrub sensitive columns if non-production users will access it.
- Keep the copy on a lower tier to control cost since test load is lighter.

Q17. You see frequent deadlocks appearing only after moving to Azure SQL Database from on-prem. How do you investigate?

Answer:

- Capture the deadlock graph from the system_health Extended Events session or via a dedicated XE session.
- Identify the conflicting resources and access order; deadlocks are logic/ordering issues, not platform-specific.
- Consider enabling Read Committed Snapshot Isolation to reduce reader/writer blocking.
- Fix access ordering, add covering indexes, and keep transactions short.

Q18. Management asks whether you should turn on automatic tuning's FORCE_LAST_GOOD_PLAN in production. What do you advise?

Answer:

- It can automatically revert a query to its last known good plan when a regression is detected — valuable for plan stability.
- Enable it after validating with Query Store data so you trust the regression detection.
- Keep CREATE INDEX/DROP INDEX automatic actions off unless you actively review recommendations.
- Monitor the tuning history so you know what the platform changed and can revert if needed.

2. Azure SQL Managed Instance — Advanced Administration

DOMAIN 2

Azure SQL Managed Instance — Advanced Administration

18 real-time scenarios

Q19. A cross-database query that worked on-prem needs to run on Managed Instance. Will it work and what do you check?

Answer:

- Managed Instance supports cross-database queries within the same instance, unlike single Azure SQL Database.
- Confirm all referenced databases live on the same MI; cross-instance still needs linked servers or other patterns.
- Check that instance-scoped features used (SQL Agent, msdb, linked servers) are configured on the MI.
- Validate collation compatibility across the databases involved.

Q20. Your MI subnet is running out of IP addresses and you need to add more instances. What went wrong and how do you fix it?

Answer:

- The delegated subnet was sized too small — MI reserves addresses for the virtual cluster and each instance.
- You cannot easily resize a subnet in use; plan a new, larger delegated subnet (a /27 minimum, larger for growth).
- Migrate or create instances in the correctly sized subnet.
- Document subnet sizing in IaC so future deployments reserve adequate address space.

Q21. You need SQL Server Agent jobs with database mail notifications on MI. What is different from on-prem?

Answer:

- SQL Agent exists on MI and supports most job types; some subsystems differ from box SQL Server.
- Database Mail is supported — configure a profile and an SMTP relay reachable from the MI subnet.
- Ensure outbound network rules allow the SMTP endpoint through the subnet's NSG/route table.
- Test a job failure alert end-to-end so on-call actually receives notifications.

Q22. A migration to MI needs the exact same SQL Server version behavior and instance-level objects. Why is MI the right target?

Answer:

- MI offers near-100% surface-area compatibility with box SQL Server including instance-scoped features.
- It supports SQL Agent, cross-database queries, CLR, linked servers, and Service Broker within the instance.
- This minimizes application and code changes versus single-database Azure SQL Database.
- You trade some PaaS simplicity for compatibility — a good fit for lift-and-shift of complex estates.

Q23. You must restore a native .bak from on-prem into MI. What is the supported path?

Answer:

- Place the backup in Azure Blob Storage and RESTORE from URL, or use the Log Replay Service for continuous restore.
- Native cross-platform restore into MI is supported; box-to-MI is a common migration route.
- For minimal downtime, use Log Replay Service to keep applying logs until cutover.
- Validate the restored databases and run consistency checks before go-live.

Q24. Your MI needs to reach an on-prem linked server over a private connection. How do you architect it?

Answer:

- MI lives in a VNet subnet, so connect the VNet to on-prem via ExpressRoute or site-to-site VPN.
- Configure the linked server and ensure name resolution (DNS) reaches the on-prem target.

- Open the required ports through NSGs, route tables, and any firewalls in the path.
- Secure the linked server credentials and prefer least-privilege service accounts.

Q25. A General Purpose MI shows higher-than-expected storage latency during index rebuilds. What is the cause and options?

Answer:

- General Purpose separates compute and remote storage, so IO-heavy operations see remote-storage latency.
- Business Critical uses local SSD and provides much lower latency plus a built-in readable replica.
- If IO latency is the bottleneck for the workload, evaluate moving to Business Critical.
- Alternatively schedule heavy maintenance in windows and tune file/IO patterns.

Q26. You need to enforce that MI only accepts connections over a private endpoint, never the public endpoint. How?

Answer:

- Disable the public endpoint on the instance so only VNet-local/private connectivity is allowed.
- Access the MI through its VNet, private endpoint, VPN, or ExpressRoute paths only.
- Lock down NSGs to permit only the required source ranges on the MI ports.
- Verify with a connection test from outside the VNet that public access is refused.

Q27. TempDB contention appears on MI under heavy load. What are your levers?

Answer:

- Confirm multiple, appropriately sized tempdb files to spread allocation contention.
- On MI, tempdb configuration is managed but can still be tuned within supported limits.
- Reduce tempdb pressure by fixing spills (bad estimates), excessive sorts, and temp table churn.
- Scale compute if the workload legitimately saturates tempdb throughput.

Q28. You want zero-downtime patching expectations set correctly for MI stakeholders. What do you tell them?

Answer:

- Microsoft patches the instance automatically; a short failover-style reconfiguration can occur.
- Configure a maintenance window so updates land in a predictable low-traffic period.
- App retry logic makes the brief reconnect transparent to users.
- Business Critical's secondary replica reduces the impact of the reconfiguration.

Q29. A vendor app needs the instance collation to match its on-prem server exactly. Can you set it on MI?

Answer:

- MI lets you choose the instance-level collation at creation time, unlike single Azure SQL Database.
- Set it correctly during provisioning — changing instance collation later is disruptive.
- Confirm database and column collations align with the app's expectations.
- Record the collation choice in your provisioning IaC for repeatability.

Q30. You need to move a database between two Managed Instances with minimal downtime. What is the approach?

Answer:

- Take a copy-only or full backup to Blob and restore to the target instance, then apply log backups.
- Or use Log Replay Service to continuously restore logs until a short cutover.
- Redirect the application connection string at cutover after final log restore.
- Validate objects, logins, and jobs on the target before switching traffic.

Q31. Logins and users don't line up after moving databases to a new MI. How do you fix orphaned users?

Answer:

- Recreate server-level logins on the new instance, ideally as Entra ID logins for centralized identity.
- Map database users to logins by SID or use ALTER USER ... WITH LOGIN to re-link.
- Script logins and permissions as part of the migration so nothing is missed.
- Test application and job service accounts before go-live.

Q32. Your MI's automated backups must be retained for seven years for compliance. What do you configure?

Answer:

- Configure long-term retention (LTR) policies to keep weekly/monthly/yearly backups for the required horizon.
- Short-term PITR retention handles recent recovery; LTR handles compliance archival.
- Verify restore from an LTR backup periodically — a backup you cannot restore is not a backup.
- Document the retention policy to satisfy auditors.

Q33. Service Broker and CLR are used by a legacy app. Are they supported on MI and what do you verify?

Answer:

- Both are supported on MI, which is a key reason to choose MI over single database for such apps.
- Verify CLR assemblies and their trust/security settings migrate correctly.
- Confirm Service Broker routes and endpoints work within the instance's networking.
- Test the end-to-end messaging flow after migration.

Q34. You must throttle a runaway workload on MI without killing the whole instance. What can you use?

Answer:

- Use Resource Governor on MI to cap CPU/memory for specific workload groups.
- Classify sessions by login/app so the offending workload is contained.
- Combine with query-level fixes — governance limits blast radius but does not fix bad queries.
- Monitor to confirm the classification is catching the intended sessions.

Q35. A DBA asks whether they can RDP into the MI host to check the OS. What do you explain?

Answer:

- No — MI is a managed PaaS instance; there is no OS/host access.
- Use DMVs, Query Store, Extended Events, and Azure Monitor for diagnostics instead.
- Instance-level configuration is done through supported T-SQL and portal/API surfaces.
- If OS-level control is a hard requirement, SQL Server on Azure VM is the correct target instead.

Q36. Cross-region DR is required for a Business Critical MI. What do you configure?

Answer:

- Set up an auto-failover group pairing the MI with an instance in the secondary region.
- This provides a read-write listener and grouped failover across regions.
- Replication is asynchronous, so plan for a small RPO on hard region loss.
- Size the secondary to carry production load and test a failover before you rely on it.

3. SQL Server on Azure Virtual Machines (IaaS) — Advanced

DOMAIN 3

SQL Server on Azure Virtual Machines (IaaS) — Advanced

15 real-time scenarios

Q37. Your SQL Server VM shows poor IO throughput. Walk through the storage tuning checklist.

Answer:

- Use Premium SSD or Ultra Disk for data/log and separate them onto different disks.
- Stripe multiple disks with Storage Spaces for higher aggregate IOPS/throughput.
- Format data/log volumes with 64 KB allocation unit size for SQL Server.
- Enable read caching on data disks and no caching on log disks; verify the VM size's disk/throughput limits are not the ceiling.

Q38. You installed via the SQL IaaS Agent extension. What benefits does that unlock?

Answer:

- Automated patching, automated backups, and flexible licensing (switch between PAYG and BYOL).
- Azure portal manageability for the SQL instance without deep VM access.
- Storage configuration best-practice application during setup.
- It is the recommended way to run SQL on Azure VM for manageability.

Q39. A three-node Always On AG on Azure VMs needs a listener that works with Azure networking. What is required?

Answer:

- Standard Azure networking does not support gratuitous ARP, so use an Azure Load Balancer (or distributed network name) for the listener.
- Configure the internal load balancer with a health probe pointing at the AG probe port.
- Place nodes across availability zones for resilience and configure the cluster accordingly.
- Validate listener failover by moving the primary and confirming client reconnection.

Q40. You need quorum resilience for a WSFC cluster backing your AG on Azure VMs. What witness do you use?

Answer:

- Use a Cloud Witness backed by an Azure Storage account — ideal for cloud/cross-zone deployments.
- It provides a lightweight tie-breaker without a third full node or file share.
- Ensure the storage account is reachable from all nodes and is in a resilient location.
- Validate quorum behavior by simulating a node loss.

Q41. Backups on a SQL VM are filling local disk. What is the recommended target?

Answer:

- Back up directly to Azure Blob Storage using BACKUP TO URL.
- This offloads capacity from the VM and provides durable, geo-redundant storage.
- Use striped URL backups for large databases to improve throughput.
- Combine with the IaaS Agent automated backup or Ola Hallengren for scheduling and retention.

Q42. You want to minimize licensing cost on a dev/test SQL Server VM. What options exist?

Answer:

- Use Azure Hybrid Benefit to apply existing licenses with Software Assurance.
- Use the Developer edition for non-production, which is free for dev/test.
- Right-size the VM and use auto-shutdown schedules for idle dev/test hours.

- Consider reserved instances for steady-state production VMs to cut compute cost.

Q43. Memory pressure appears on a SQL VM sharing resources with the OS. How do you configure it?

Answer:

- Set max server memory so SQL leaves headroom for the OS and other services.
- Enable Lock Pages in Memory for the SQL service account to prevent paging of the buffer pool.
- Size the VM so the working set fits in RAM to avoid excessive IO.
- Monitor Page Life Expectancy and memory grants to validate the settings.

Q44. You need consistent, application-consistent backups of the whole VM plus SQL. What do you use?

Answer:

- Use Azure Backup for SQL Server in Azure VM for application-consistent, log-aware backups.
- It understands SQL, supports PITR, and centralizes retention policy.
- Alternatively manage native BACKUP TO URL with your own scheduling for finer control.
- Test restores regularly and document RTO/RPO expectations.

Q45. An AG secondary on an Azure VM is lagging during business hours. How do you diagnose?

Answer:

- Check the redo queue and log send queue on the secondary via AG DMVs/dashboard.
- Determine if it is network throughput between zones/regions or secondary IO/CPU saturation.
- For synchronous replicas, lag can throttle the primary — confirm commit mode expectations.
- Right-size the secondary disks/VM or move to async if the distance/throughput demands it.

Q46. You must patch the OS and SQL on AG nodes with no downtime. What is the runbook order?

Answer:

- Patch a secondary first, validate health, then fail over to it so it becomes primary.
- Patch the former primary (now secondary), validate, then optionally fail back.
- Keep synchronous commit and confirm the AG is healthy/synchronized before each failover.
- Automate the sequence and include health checks and rollback steps.

Q47. Tempdb on a SQL VM should use the local ephemeral disk. Why and how?

Answer:

- The temporary/ephemeral local SSD offers high throughput and tempdb does not need to persist across deallocation.
- Place tempdb data/log on the temp drive and script re-creation of the folder on startup.
- Ensure a startup mechanism recreates the tempdb directory since the temp disk can be wiped on deallocation.
- Size tempdb files appropriately and use multiple files to reduce contention.

Q48. Your SQL VM must not be exposed to the public internet at all. How do you lock it down?

Answer:

- Remove any public IP and restrict access to the VNet via private connectivity.
- Use NSGs to allow only required source ranges on the SQL port (avoid 1433 exposed broadly).
- Access management via Azure Bastion or a jump host instead of public RDP.
- Use Just-In-Time VM access to open management ports only when needed.

Q49. You need higher IOPS than a single Premium disk provides for a heavy write log. What do you do?

Answer:

- Use Ultra Disk for the log if you need configurable high IOPS/throughput and low latency.
- Or stripe multiple Premium disks, keeping the log on its own volume.
- Confirm the VM size supports the required uncached disk throughput — the VM cap can be the real limit.

- Measure with real workload before and after the change.

Q50. A finance team wants the SQL VM to auto-scale compute on schedule. What is realistic on IaaS?

Answer:

- VMs do not auto-scale vertically live; you resize (which requires a reboot) on a schedule via automation.
- Schedule a resize up before peak and down after using Automation runbooks or Azure Functions.
- For true elastic scaling without reboots, a PaaS tier (SQL DB/MI Serverless/Hyperscale) is the better fit.
- Communicate the brief reboot window to stakeholders for scheduled resizes.

Q51. You must ensure AG nodes survive a datacenter (zone) failure. How do you place them?

Answer:

- Distribute AG replicas across availability zones within the region.
- Use a Cloud Witness for quorum so a zone loss does not break the cluster.
- Confirm the load-balancer/listener spans zones and health probes work across them.
- Test by simulating a zone-level node loss and confirming failover and quorum hold.

4. High Availability & Disaster Recovery — Advanced

DOMAIN 4

High Availability & Disaster Recovery — Advanced

22 real-time scenarios

Q52. Your failover group failed over automatically, but the app still points at the old primary. What went wrong?

Answer:

- The app is using a direct database endpoint instead of the failover group's read-write listener.
- Always connect through the FOG listener FQDN so failover redirects transparently.
- Update connection strings and validate DNS resolves the listener to the current primary.
- Re-test a failover to confirm the app follows without manual changes.

Q53. You need to run a real DR drill without risking production. How do you do it safely?

Answer:

- Use a planned (friendly) failover on the failover group so no data is lost.
- Validate the app connects, key transactions work, and monitoring follows the new primary.
- Fail back after the drill and document RTO/RPO actually achieved versus targets.
- Schedule drills regularly — an untested DR plan is an assumption, not a plan.

Q54. Explain the difference between active geo-replication and auto-failover groups to a stakeholder choosing between them.

Answer:

- Geo-replication creates readable secondaries you fail over individually with your own endpoint management.
- Failover groups add a stable read-write/read-only listener and group multiple databases for one failover.
- Failover groups can fail over automatically on region issues; geo-replication is more manual.
- Choose failover groups for app-transparent DR; geo-replication for granular read-scale control.

Q55. During a forced failover you accepted potential data loss. How do you quantify and communicate what was lost?

Answer:

- A forced failover on async replication can lose transactions not yet replicated — the RPO window.
- Estimate loss from replication lag metrics captured before the event.
- Reconcile downstream systems and communicate the affected time window to stakeholders.
- Prefer planned failover whenever the primary is still reachable to avoid loss.

Q56. A Business Critical database's built-in replica isn't enough; you need cross-region DR too. How do they combine?

Answer:

- Business Critical provides local HA replicas within the region automatically (in-region resilience).
- Add a failover group or geo-replica for cross-region DR on top of that local HA.
- Local replicas cover node/zone failures; the geo secondary covers region loss.
- Test both layers independently so you know each works.

Q57. Your RTO is 15 minutes but geo-restore takes hours for your database size. How do you close the gap?

Answer:

- Geo-restore is a last resort and is slow because it seeds from geo-redundant backups.
- Use a warm secondary (failover group/geo-replication) so failover is near-instant instead of restoring.
- For very large databases, Hyperscale's snapshot-based restore dramatically cuts restore time.

- Match the DR mechanism to the RTO — backups alone rarely meet tight RTOs.

Q58. You need read traffic served locally in a DR region during normal operations, not just at failover. What do you use?

Answer:

- Use geo-replication/failover group secondaries which are readable during normal operation.
- Route read-only workloads regionally to the nearest secondary.
- Confirm the app tolerates replication lag (eventual consistency) for those reads.
- This turns your DR investment into everyday read-scale value.

Q59. A failover group contains 10 databases but one app only needs 2 of them failed over. What's the design flaw?

Answer:

- Failover groups fail over as a unit — grouping unrelated databases couples their DR lifecycle.
- Separate databases into failover groups aligned to application/DR boundaries.
- This lets you fail over just the app that needs it without dragging others along.
- Design groups around blast radius and recovery ownership.

Q60. How do you validate that your secondary can actually carry production load after failover?

Answer:

- Size the secondary at the same tier/compute as the primary — an undersized secondary fails under real load.
- During a DR drill, run representative production load and watch resource utilization.
- Confirm connection limits, worker threads, and IO headroom on the secondary.
- Adjust sizing before you need it in a real event.

Q61. Your on-prem AG needs to extend a DR replica into Azure. What are the moving parts?

Answer:

- Extend the WSFC/AG to an Azure VM replica over ExpressRoute or VPN for a hybrid AG.
- Use async commit for the cross-site replica to avoid throttling the on-prem primary.
- Handle the listener with an Azure Load Balancer for the Azure-side replica.
- Plan quorum carefully so a site loss does not break the cluster.

Q62. After a failover, monitoring and alerting went silent. What did you forget?

Answer:

- Alerts/monitoring were bound to the old primary resource instead of the listener/failover group.
- Configure monitoring against the stable listener and both replicas.
- Ensure diagnostic settings and alert rules exist on the secondary too.
- Validate alerting during DR drills, not only in production steady-state.

Q63. A stakeholder asks: what's our realistic RPO and RTO for a full region outage? How do you answer responsibly?

Answer:

- RPO depends on replication lag at the moment of failure (async = seconds of potential loss).
- RTO depends on the mechanism: failover group failover is minutes; geo-restore is much longer.
- State the numbers you have actually measured in drills, not marketing figures.
- Recommend the architecture that meets the agreed targets and note the cost trade-off.

Q64. You want automatic failover only after sustained region trouble, not brief blips. How do you tune it?

Answer:

- Set the failover group's grace period so automatic failover only triggers after the outage persists.

- Too short causes unnecessary failovers on transient issues; too long hurts RTO.
- Balance the grace period against your RTO commitment.
- Combine with app retry logic to ride out short blips without failing over.

Q65. Your DR plan assumes DNS changes propagate instantly. Why is that risky and what do you do?

Answer:

- Client DNS caching can delay clients picking up the new endpoint after failover.
- Use the failover group listener whose DNS is managed to point at the current primary.
- Keep TTLs sensible and ensure clients reconnect rather than caching a stale IP.
- Test client reconnection timing during drills.

Q66. A database must never lose a single committed transaction. Is that achievable in Azure and at what cost?

Answer:

- True zero-RPO across regions is not guaranteed with async geo-replication.
- In-region Business Critical uses synchronous replicas for zero data loss within the region.
- Cross-region, you accept a small RPO or design app-level idempotency/reconciliation.
- Set expectations honestly and design compensating controls for the residual risk.

Q67. You need to fail over a whole application stack (multiple databases + app tier) together. How do you coordinate?

Answer:

- Group the databases in a failover group so they move together.
- Coordinate app-tier failover (Traffic Manager/Front Door) with the database failover.
- Script the sequence so database and app cutover happen in the right order.
- Rehearse the full-stack failover, not just the database layer.

Q68. Replication lag spiked and your synchronous replica started slowing writes. What are your options?

Answer:

- Synchronous commit waits for the secondary, so secondary/network problems throttle the primary.
- Diagnose the secondary IO/CPU or network bottleneck causing the lag.
- Temporarily switch to async if availability of the primary matters more than zero-loss right now.
- Fix the underlying bottleneck, then return to sync for zero data loss.

Q69. How do you decide between zone-redundant HA and geo (region) DR for a given database?

Answer:

- Zone-redundancy protects against a datacenter/zone failure within one region — cheap, low latency.
- Geo DR protects against a whole-region loss — higher cost and cross-region latency.
- Most critical systems want both: zone-redundant HA plus a geo secondary.
- Decide based on the failure domains your SLA must survive.

Q70. A team enabled auto-failover but never tested failback. Why is that a problem?

Answer:

- Failover is only half the cycle — returning to the primary region needs its own validated process.
- Failback re-seeds/reverses roles and can surprise you if untested.
- Include failback in DR drills and document the steps and timing.
- Confirm data reconciliation after failback if any loss occurred.

Q71. Your compliance team asks for evidence that DR works. What do you provide?

Answer:

- Provide DR drill records: dates, RTO/RPO achieved, steps executed, and issues found.

- Include failover group configuration and monitoring evidence.
- Show restore-test results for backups (LTR restores validated).
- Maintain a runbook and change history as auditable artifacts.

Q72. A single logical server hosts many databases; you fail over the server-level failover group. What server-level objects must exist on the secondary?

Answer:

- Server-level logins, firewall rules, and auditing settings do not automatically replicate — recreate them on the secondary server.
- Only database contents replicate; server-scoped configuration is your responsibility.
- Script these into IaC so the secondary is a true equivalent.
- Validate app logins work post-failover during drills.

Q73. You need DR for a Hyperscale database. What's different about its options?

Answer:

- Hyperscale supports geo-replicas and failover groups, with fast restore due to its storage architecture.
- Its snapshot-based storage makes seeding and restore far quicker than traditional tiers.
- Confirm the specific HA/replica options for your Hyperscale configuration and region.
- Test failover as with any other tier to confirm behavior.

5. Backup, Restore & Recovery

DOMAIN 5

Backup, Restore & Recovery

16 real-time scenarios

Q74. A developer dropped a table 30 minutes ago in Azure SQL Database. What's the fastest recovery with least impact?

Answer:

- Use point-in-time restore to a new database at a timestamp just before the drop.
- PITR always restores to a new database — it never overwrites the live one.
- Extract just the dropped table from the restored copy and merge it back.
- Avoid restoring the whole production database over itself for a single-object mistake.

Q75. You need to prove you can meet a 1-hour RTO from backups. How do you validate it?

Answer:

- Run an actual timed restore of a representative database and measure end-to-end duration.
- Restore time scales with size and tier — test the largest critical database.
- If restore exceeds RTO, move to warm-standby DR instead of relying on restore.
- Repeat the test periodically and after major size growth.

Q76. Compliance requires yearly backups kept for 10 years. What do you configure and how do you keep cost down?

Answer:

- Configure long-term retention (LTR) with a yearly retention rule for the required horizon.
- LTR backups sit in cheaper storage separate from short-term PITR backups.
- Only retain the cadence you actually need (e.g., one yearly) to control cost.
- Periodically restore an LTR backup to confirm it is usable.

Q77. You restored a PITR copy but the app can't log in to it. Why?

Answer:

- The restored database is a new database; server-level logins/users may not map automatically.
- Recreate or re-map logins and fix orphaned users on the restored database.
- Grant the app's principal the needed permissions on the new database.
- Script login/permission recreation as part of your restore runbook.

Q78. How far back can you restore an Azure SQL Database by default, and how do you extend it?

Answer:

- PITR retention defaults to a limited window (commonly up to 7 days, configurable up to 35).
- Increase PITR retention within the supported maximum for longer recent recovery.
- For anything older, rely on long-term retention backups.
- Match retention to your recovery requirements and audit needs.

Q79. You need to restore a database into a different region entirely. What's the mechanism?

Answer:

- Use geo-restore, which restores from geo-redundant backups into any region.
- Expect longer restore time since it seeds from geo-replicated backup storage.
- For faster cross-region recovery, keep a geo-replica/failover group instead.
- Validate the restored database and re-point the app.

Q80. A large database's automated backups seem to affect performance during business hours. What do you check?

Answer:

- Managed backups are designed to be low-impact and platform-scheduled — confirm the perceived impact is real.
- Correlate with Query Store/resource metrics to see if backups or the workload cause the load.
- For IaaS, schedule native backups off-peak and use striped URL backups.
- Ensure backup storage/network throughput is not a bottleneck for VM-based backups.

Q81. You must restore only a few rows accidentally updated, not the whole table. What's the cleanest approach?

Answer:

- Restore a PITR copy to a new database, then selectively merge the correct rows back.
- If the table is temporal, query the row history directly — no restore needed.
- Apply the correction in a controlled transaction with a backup of current state.
- Prefer surgical fixes over wholesale restores to limit blast radius.

Q82. How do you back up a database before a risky release so you can roll back instantly?

Answer:

- In Azure SQL Database, note the exact pre-release timestamp — PITR is your rollback point.
- Optionally create a database copy as a fixed, named rollback snapshot.
- For MI/VM, take a copy-only backup before the change.
- Document the exact rollback procedure and test the assumption once.

Q83. Your LTR restore is needed for an audit but nobody has tested it. What's your process?

Answer:

- Restore the specific LTR backup to a new database in a non-production context.
- Validate contents and provide the auditor evidence without touching production.
- Time the restore so you know the real recovery duration for that vintage.
- Add periodic LTR restore tests to your operational calendar.

Q84. A restored copy is on an expensive tier by default. How do you control restore cost?

Answer:

- Restore to an appropriate (often lower) service tier for the recovery purpose.
- Scale it down further if it is only for extraction/validation, not production load.
- Delete the restore copy promptly once the recovery is complete.
- Automate cleanup so orphaned restore copies do not accumulate cost.

Q85. You need application-consistent backups for a multi-database transaction boundary. Is PITR enough?

Answer:

- PITR restores each database to a point in time but does not coordinate cross-database consistency by itself.
- Restore all involved databases to the same timestamp and reconcile at the app boundary.
- Design idempotent/compensating logic for cross-database operations.
- Document the recovery order and consistency expectations.

Q86. How do you handle backup retention when a database is deleted — can you still recover it?

Answer:

- A deleted database can be restored from its backups within the retention window (deleted-database restore).
- Act within the retention period — after it lapses, recovery is not possible.
- For long-lived recoverability, ensure LTR was configured before deletion.
- Protect against accidental deletion with resource locks and RBAC.

Q87. Your VM-based SQL backups to URL keep failing intermittently. What do you investigate?

Answer:

- Check the storage account access (SAS/managed identity), network path, and throttling.
- Use striped backups and validate the container permissions.
- Confirm the VM's outbound rules allow the storage endpoint.
- Add retry and alerting so failures are caught, and verify restores succeed.

Q88. You want a single, auditable backup policy across many databases. How do you standardize?

Answer:

- Use Azure Policy and consistent PITR/LTR settings applied via IaC across databases.
- Centralize monitoring of backup health and retention compliance.
- For VMs, standardize on Azure Backup for SQL with a shared policy.
- Report on compliance so gaps are visible to auditors.

Q89. A restore succeeded but performance is terrible on the new copy. What's a likely cause?

Answer:

- Statistics or plan cache differences and a lower target tier can cause slow initial performance.
- Update statistics and let the workload warm the cache.
- Confirm the restore target tier matches the workload's needs.
- Check for missing indexes if the copy is on a different configuration.

6. Performance Tuning & Query Optimization

DOMAIN 6

Performance Tuning & Query Optimization

22 real-time scenarios

Q90. A query regressed overnight with no code change. How do you find and fix the cause quickly?

Answer:

- Open Query Store, compare plans for the query across time, and identify the regressed plan.
- Force the last known good plan to stabilize immediately.
- Investigate the root cause — stats change, parameter sniffing, or data growth.
- Update statistics or add hints/indexes for a durable fix, then unforce if appropriate.

Q91. You suspect parameter sniffing is causing wildly varying performance. How do you confirm and address it?

Answer:

- Compare plans for different parameter values in Query Store — a single cached plan hurting some values indicates sniffing.
- Options: OPTIMIZE FOR, RECOMPILE, or forcing a representative plan.
- Consider redesigning the query or using filtered indexes for skewed data.
- Validate the fix across the full range of parameter values.

Q92. CPU is pinned at 100% on Azure SQL Database. Walk through your triage.

Answer:

- Identify top CPU-consuming queries via Query Store / sys.dm_exec_query_stats.
- Look for missing indexes, implicit conversions, and bad plans driving scans.
- Tune the worst offenders before scaling the tier.
- If the workload legitimately needs more CPU, scale up — but only after tuning.

Q93. A report query does huge logical reads. How do you reduce them?

Answer:

- Add a covering index so the query seeks and returns from the index without key lookups.
- Eliminate scans by making predicates SARGable (no functions on columns).
- Reduce returned columns/rows to what is actually needed.
- Verify improvement by comparing logical reads before and after.

Q94. You see high PAGELATCH waits on tempdb-heavy workloads. What do you do?

Answer:

- Ensure multiple, equally sized tempdb data files to spread allocation-page contention.
- Reduce tempdb usage from spills and excessive temp tables/table variables.
- Fix cardinality estimate problems causing spills to tempdb.
- Scale compute only if contention persists after tuning.

Q95. Automatic tuning recommends an index but you're cautious. How do you evaluate it?

Answer:

- Review the recommendation's estimated impact and the query it targets.
- Check it does not duplicate an existing index or bloat write cost.
- Test in a copy under representative load before applying to production.
- Apply, then monitor Query Store to confirm the expected improvement.

Q96. A query is fast for small inputs but times out for large ones. What pattern is this and the fix?

Answer:

- Likely a plan that does not scale (nested loops where a hash/merge is better) due to estimates.
- Update statistics and check for skew; ensure the optimizer sees accurate cardinality.
- Consider indexing to support the large-input access path.
- Use hints only as a last resort after fixing estimates.

Q97. You need to find which queries consume the most resources over the last week. Where do you look?

Answer:

- Query Store's Top Resource Consumers and query-level metrics over the retention window.
- Sort by CPU, duration, logical reads, and memory grants to find offenders.
- Drill into plans and execution counts to prioritize fixes by total impact.
- Fix high-frequency moderate queries — they often beat rare heavy ones in total cost.

Q98. Blocking chains appear during peak load. How do you break them?

Answer:

- Identify the head blocker via sys.dm_exec_requests / blocking DMVs.
- Shorten transactions, fix missing indexes causing scans, and access objects in a consistent order.
- Consider Read Committed Snapshot Isolation to reduce reader/writer blocking.
- Address the root query, not just the symptom, to prevent recurrence.

Q99. A columnstore report is slower than expected. What do you check?

Answer:

- Confirm rowgroups are healthy (not too many open/trim-reason issues) and well-compressed.
- Ensure segment elimination is working via ordered data / good predicates.
- Rebuild/reorganize to clean up deleted rows and small rowgroups.
- Verify batch-mode execution is actually being used.

Q100. Memory grants are too large and cause spills or waits. How do you address it?

Answer:

- Investigate over-estimated grants from bad cardinality estimates.
- Update statistics and consider Memory Grant Feedback (intelligent query processing).
- Right-size queries returning excessive columns/rows.
- Monitor RESOURCE_SEMAPHORE waits to confirm improvement.

Q101. You want to reduce recompilations hurting a hot procedure. What do you look at?

Answer:

- Check for RECOMPILE hints, frequent stats updates, or schema changes forcing recompiles.
- Stabilize with plan forcing if a good plan exists.
- Avoid patterns (temp table churn, SET option changes) that invalidate plans.
- Measure compile time vs execution to confirm it is worth addressing.

Q102. A join between a huge and a tiny table scans the huge one. How do you fix it?

Answer:

- Ensure an index supports the join/seek predicate on the large table.
- Confirm predicates are SARGable and statistics are current.
- Check the plan chose an appropriate join type given real cardinalities.
- Consider filtered indexes if only a subset is queried.

Q103. Intelligent Query Processing features could help but you're on an older compatibility level. What do you do?

Answer:

- Raise the database compatibility level to unlock IQP features like adaptive joins and memory grant feedback.
- Test the workload at the new level using Query Store to catch any regressions.
- Use plan forcing to protect any query that regresses during the transition.
- Roll forward gradually with monitoring rather than flipping blindly.

Q104. You need to throttle a noisy tenant's queries in a shared database. What's available?

Answer:

- On MI/VM, use Resource Governor to cap the tenant's workload group.
- On single database, isolate heavy tenants into their own database or elastic pool.
- Add query-level limits and app-side rate limiting.
- Monitor to confirm the noisy neighbor is actually contained.

Q105. Statistics seem stale on a fast-growing table. How do you manage updates without heavy overhead?

Answer:

- Rely on auto-update stats but add scheduled targeted updates for large, volatile tables.
- Use sampled updates for very large tables to bound the cost.
- Watch for the threshold where auto-update triggers too infrequently on big tables.
- Validate plan quality improves after the update.

Q106. A query uses a scalar UDF and is slow row-by-row. What's the modern fix?

Answer:

- Scalar UDF inlining (recent compatibility levels) can dramatically improve performance.
- Raise compatibility level to enable inlining and test the query.
- Alternatively rewrite the UDF logic inline or as an inline table-valued function.
- Confirm the plan no longer shows per-row function execution.

Q107. You want to catch regressions before users do. What proactive setup do you use?

Answer:

- Enable Query Store with sensible retention and capture settings on all critical databases.
- Alert on plan regressions and rising resource consumption.
- Turn on automatic plan correction to auto-revert clear regressions.
- Review Top Consumers weekly as a habit, not only during incidents.

Q108. Adaptive query processing chose a bad join for a skewed workload. How do you handle it?

Answer:

- Investigate the skew and whether statistics represent it accurately.
- Use filtered statistics/indexes for skewed value ranges.
- Force a plan for the problematic pattern if needed.
- Re-evaluate after data distribution changes.

Q109. A nightly ETL saturates the database and hurts morning users. How do you smooth it?

Answer:

- Schedule ETL in a true off-peak window and throttle its batch size.
- Use minimally logged patterns and partition switching for large loads.
- Consider Serverless auto-scaling or a temporary tier bump during the load.
- Isolate ETL onto a separate replica/database if it keeps colliding with OLTP.

Q110. You need to compare workload performance before and after a tier change objectively. How?

Answer:

- Use Query Store to capture baselines (duration, CPU, reads) before the change.
- Apply the change and compare the same queries over an equivalent window.
- Watch dominant wait types to confirm the bottleneck actually moved.
- Document the delta so the decision is evidence-based.

Q111. A query intermittently gets a bad plan after stats updates. How do you stabilize it?

Answer:

- Force the known good plan in Query Store so stats updates don't destabilize it.
- Investigate why the new plan is chosen (estimate swings, skew).
- Consider OPTIMIZE FOR or a filtered index for the volatile predicate.
- Unforce once the root cause is fixed and validated.

7. Security, Compliance & Data Protection

DOMAIN 7

Security, Compliance & Data Protection

20 real-time scenarios

Q112. Auditors require that DBAs cannot read sensitive PII columns even though they manage the database. How?

Answer:

- Use Always Encrypted so plaintext is only visible to the client with the keys, not to the DBA or the engine.
- Store keys in a client-controlled key store (Key Vault) separate from DBA access.
- Combine with dynamic data masking for casual obfuscation and RBAC for least privilege.
- Document the separation-of-duties control for the audit.

Q113. You must eliminate SQL authentication and use centralized identity. What do you implement?

Answer:

- Use Microsoft Entra ID authentication and set an Entra admin on the server/instance.
- Create contained database users mapped to Entra identities/groups.
- Disable or minimize SQL logins; manage access via Entra group membership.
- Enable MFA and conditional access at the identity layer.

Q114. A compliance rule says data must be encrypted at rest with a key you control and can rotate. What do you configure?

Answer:

- Enable TDE with a customer-managed key (BYOK) stored in Azure Key Vault.
- You control rotation and can revoke the key to render data inaccessible.
- Ensure the server's managed identity has Key Vault access and soft-delete/purge protection is on.
- Test rotation and monitor key access.

Q115. You need to restrict database access to a specific corporate network only. What layers apply?

Answer:

- Use private endpoints/VNet integration so traffic never traverses the public internet.
- Disable public network access on the server/instance.
- Add firewall rules or NSGs scoped to the corporate ranges as defense in depth.
- Verify external connectivity is refused.

Q116. A row-level access requirement: users must only see their own region's rows. How do you enforce it in the database?

Answer:

- Implement Row-Level Security with a security policy and predicate function keyed on the user's context.
- Set the session context (e.g., region) securely at connection time.
- Test that filtering cannot be bypassed by direct queries.
- Combine with least-privilege permissions for defense in depth.

Q117. Sensitive columns should be masked for support staff but visible to the app. What's the lightweight option?

Answer:

- Use Dynamic Data Masking to obfuscate columns for users without UNMASK permission.
- Grant the app principal UNMASK; withhold it from support roles.

- Remember DDM is obfuscation, not encryption — pair it with RLS/permissions for real protection.
- Verify masked output for the support role.

Q118. You must detect anomalous or malicious database activity automatically. What do you turn on?

Answer:

- Enable Microsoft Defender for SQL for threat detection (SQL injection, anomalous access, exfiltration patterns).
- Route alerts to security operations and integrate with your SIEM.
- Enable auditing to a Log Analytics workspace for investigation.
- Review and tune alerts to reduce noise.

Q119. A vulnerability assessment flagged excessive permissions. How do you remediate systematically?

Answer:

- Run the SQL Vulnerability Assessment and work the findings by severity.
- Apply least privilege — remove broad roles, scope permissions to what's needed.
- Set an assessment baseline so accepted items don't re-alert.
- Re-scan to confirm remediation and track over time.

Q120. You need to prove exactly who accessed a table and when for the last 90 days. What's the setup?

Answer:

- Enable auditing to Log Analytics/storage capturing the relevant actions with sufficient retention.
- Query the audit logs for access to the specific object.
- Protect audit configuration so it can't be silently disabled (RBAC).
- Retain logs per the compliance horizon.

Q121. Keys in Key Vault must never be exposed and access must be logged. How do you harden it?

Answer:

- Enable Key Vault logging/diagnostics and restrict access with RBAC and access policies.
- Turn on soft-delete and purge protection to prevent key loss.
- Use managed identities for services to avoid stored secrets.
- Monitor key access and alert on anomalies.

Q122. A pen test wants to confirm data in transit is encrypted. What do you verify?

Answer:

- Enforce Encrypt=true / trusted TLS on connections; Azure SQL requires encryption in transit.
- Validate the minimum TLS version enforced on the server/instance.
- Confirm clients don't disable certificate validation.
- Document the TLS posture for the report.

Q123. You must ensure deleted key access renders backups unreadable too. Does BYOK cover that?

Answer:

- With TDE + customer-managed key, revoking the key makes the database (and its encrypted backups) inaccessible.
- This gives you a cryptographic kill-switch for compliance.
- Guard the key lifecycle carefully — losing it means losing the data.
- Test the revoke/restore-access flow in a non-production setting.

Q124. Separation of duties: the security team owns keys, DBAs own operations. How do you structure this?

Answer:

- Security team manages Key Vault and rotation; DBAs operate the database without key access.
- Use RBAC to enforce the boundary; the server identity accesses the key, not the DBA.
- Always Encrypted keeps sensitive plaintext away from DBAs entirely.

- Document the control matrix for auditors.

Q125. A third party needs temporary read access to one schema. How do you grant it safely?

Answer:

- Create a dedicated least-privilege principal scoped to only that schema's read permissions.
- Time-box the access and remove it on schedule.
- Prefer Entra identity with conditional access over a shared SQL login.
- Audit their activity for the duration.

Q126. You need to classify which columns are sensitive across many databases. What tool helps?

Answer:

- Use Data Discovery & Classification to label sensitive columns (PII, financial, health).
- Classification feeds auditing and reporting on sensitive-data access.
- Standardize labels and review classification coverage.
- Use the labels to drive masking/RLS decisions.

Q127. An app connection string still has a password in plaintext config. How do you fix it?

Answer:

- Switch to a managed identity so no secret is stored in the app config.
- If a secret is unavoidable, store it in Key Vault and reference it at runtime.
- Rotate the exposed credential immediately.
- Scan configs/repos to ensure no secrets remain.

Q128. Compliance mandates that database changes are reviewed and traceable. How do you enforce it?

Answer:

- Manage schema via source control and CI/CD with mandatory review/approval gates.
- Enable auditing to record who applied changes and when.
- Restrict direct production DDL to the pipeline's service principal.
- Keep change history as an auditable artifact.

Q129. You suspect a SQL injection attempt was made. How do you detect and respond?

Answer:

- Defender for SQL flags potential injection patterns — review the alert details.
- Check audit logs for the offending statements and source.
- Fix the vulnerable app code (parameterize queries) as the real remediation.
- Rotate any exposed credentials and tighten least privilege.

Q130. Backups themselves must be protected from tampering/deletion. How do you secure them?

Answer:

- Use immutable/soft-delete-protected storage and RBAC on backup resources.
- Restrict who can change retention or delete backups.
- For LTR, rely on the platform's managed, access-controlled backup storage.
- Monitor and alert on backup deletion attempts.

Q131. A regulator asks how you'd revoke all access instantly if the estate is compromised. What's your answer?

Answer:

- Disable the affected identities/logins and rotate/revoke keys (TDE BYOK kill-switch).
- Cut network access via firewall/private-endpoint changes.
- Fail to an isolated known-good copy if integrity is in question.
- Have this incident-response runbook written and rehearsed in advance.

8. Monitoring, Alerting & Diagnostics

DOMAIN 8

Monitoring, Alerting & Diagnostics

16 real-time scenarios

Q132. You want a single pane of glass for many databases' health. What do you build?

Answer:

- Send metrics/diagnostics to a Log Analytics workspace and build Azure Monitor workbooks.
- Aggregate CPU, IO, waits, deadlocks, and connection errors across the estate.
- Add alert rules on the key thresholds.
- Share the workbook so stakeholders see the same truth.

Q133. An intermittent slowdown happens at random times. How do you catch it in the act?

Answer:

- Enable an Extended Events session (or use system_health) targeting the suspected events.
- Capture waits, long-running queries, and blocking around the incident window.
- Correlate with Azure Monitor metrics for resource saturation.
- Review Query Store for the plans active during the slowdown.

Q134. You need alerting on deadlocks so developers can fix the code. How?

Answer:

- Capture deadlock graphs (system_health XE / audit) and alert when they occur.
- Route the graph to the dev team with the conflicting statements.
- Track deadlock frequency over time to measure improvement.
- Consider RCSI to reduce reader/writer conflicts.

Q135. Connection failures spiked. How do you tell throttling from a real outage?

Answer:

- Check the specific error numbers — throttling (e.g., 40501) differs from availability errors.
- Correlate with resource metrics (DTU/vCore, sessions, workers) for saturation.
- Check Service Health/Resource Health for platform events.
- Add retry logic so transient throttling doesn't surface to users.

Q136. You want to know when a database is trending toward its storage limit before it hits it. What do you set?

Answer:

- Alert on used-space percentage crossing a threshold below the cap.
- Trend storage growth in a workbook to forecast the exhaustion date.
- Automate scaling or cleanup when the threshold trips.
- Never let a hard max-size stop catch you by surprise.

Q137. Which wait statistics do you watch to characterize a workload's bottleneck?

Answer:

- CPU (SOS_SCHEDULER_YIELD), IO (PAGEIOLATCH), log (WRITELOG/LOG_RATE_GOVERNOR), memory (RESOURCE_SEMAPHORE), locking (LCK_*).
- The dominant wait tells you where to focus tuning or scaling.
- Track waits over time, not just a snapshot.
- Confirm the wait actually shifts after a change.

Q138. You need proactive alerting on query regressions, not just resource metrics. What do you use?

Answer:

- Use Query Store data and automatic tuning's regression detection.
- Alert on new plans with worse resource profiles for critical queries.
- Enable FORCE_LAST_GOOD_PLAN to auto-mitigate clear regressions.
- Review the tuning history regularly.

Q139. A stakeholder wants a weekly performance report without manual effort. How do you deliver it?

Answer:

- Build an Azure Monitor workbook and schedule it, or push metrics to a dashboard.
- Include top consumers, waits, errors, and capacity trend.
- Automate delivery (email/Teams) via a Logic App or Automation.
- Keep it concise and action-oriented.

Q140. You must detect when automated failover happened and alert on-call. How?

Answer:

- Alert on failover-group role changes and Resource Health events.
- Correlate with connection error spikes to confirm impact.
- Ensure alerts point at the listener, not just the old primary.
- Test the alert during DR drills.

Q141. How do you monitor a Managed Instance's SQL Agent job failures centrally?

Answer:

- Configure job failure notifications via Database Mail and operators.
- Also surface msdb job history to Log Analytics for central alerting.
- Alert on consecutive failures and long-running jobs.
- Validate on-call actually receives the alert.

Q142. You want to baseline 'normal' so anomalies stand out. What's the approach?

Answer:

- Capture metrics over a representative period to establish baselines per database.
- Use dynamic thresholds in Azure Monitor to flag deviations automatically.
- Revisit baselines after major workload changes.
- Alert on deviation, not just static thresholds.

Q143. A database is slow but metrics look fine. Where else do you look?

Answer:

- Check waits and blocking — a single blocker can stall users without pegging resources.
- Look at Query Store for a specific regressed query.
- Check client-side/network latency and connection pooling issues.
- Confirm the slowness is server-side before scaling.

Q144. You need to retain diagnostic logs for a year for compliance. How?

Answer:

- Route diagnostic settings to a Log Analytics workspace and/or storage with the required retention.
- Set retention policies to meet the compliance horizon.
- Control access to the logs and monitor for tampering.
- Validate you can query historical logs when needed.

Q145. Alert fatigue is causing on-call to ignore pages. How do you fix the alerting?

Answer:

- Tune thresholds and use dynamic baselines to cut false positives.
- Consolidate correlated alerts and set severities meaningfully.
- Ensure each alert is actionable with a runbook link.
- Review alert quality regularly and retire noisy rules.

Q146. You want to catch long-running or blocking queries automatically. What's a lightweight method?

Answer:

- Schedule a query against DMVs (sys.dm_exec_requests) to capture long/blocked sessions and alert.
- Or use an Extended Events session for blocking beyond a duration threshold.
- Notify with the offending statement and blocking chain.
- Feed patterns back to developers for permanent fixes.

Q147. How do you monitor cost as a first-class metric, not just performance?

Answer:

- Use Cost Management with budgets and alerts per resource/resource group.
- Tag databases by owner/app for cost allocation.
- Trend spend and correlate with tier/scaling changes.
- Review right-sizing recommendations periodically.

9. Migration & Modernization

DOMAIN 9

Migration & Modernization

16 real-time scenarios

Q148. You must choose between Azure SQL Database, MI, and SQL on VM for a legacy app. How do you decide?

Answer:

- Map required features: instance-scoped features (Agent, cross-db, CLR, linked servers) point to MI.
- Full OS/instance control or unsupported features point to SQL on VM.
- Simple, single-database apps fit Azure SQL Database with least management.
- Assess compatibility with Data Migration Assistant before committing.

Q149. How do you assess migration blockers before moving to Azure SQL Database?

Answer:

- Run Data Migration Assistant to find unsupported features and breaking changes.
- Review deprecated syntax, cross-database dependencies, and agent jobs.
- Remediate or choose a more compatible target (MI) based on findings.
- Re-assess after remediation.

Q150. You need near-zero-downtime migration from on-prem to Azure SQL MI. What do you use?

Answer:

- Use Azure Database Migration Service in online mode or the Log Replay Service.
- Continuously sync changes until a short cutover window.
- Validate data and objects on the target before switching the app.
- Rehearse the cutover to know its real duration.

Q151. A migration must be reversible if go-live fails. How do you plan rollback?

Answer:

- Keep the source running read/write until cutover; don't decommission early.
- Have a documented switch-back of connection strings.
- Capture a clean pre-cutover state to return to.
- Define go/no-go criteria and a rollback trigger.

Q152. You're migrating many databases in waves. How do you keep it organized and low-risk?

Answer:

- Group by app/blast radius and migrate in waves with validation gates.
- Automate the repeatable steps (assessment, migration, validation) via runbooks.
- Start with low-risk databases to prove the process.
- Track status centrally and communicate cutover windows.

Q153. How do you migrate SQL Server logins and permissions without breaking apps?

Answer:

- Script server logins and re-create them (prefer Entra identities) on the target.
- Fix orphaned users by mapping SIDs or ALTER USER ... WITH LOGIN.
- Migrate role memberships and object permissions with the schema.
- Test app/service accounts before go-live.

Q154. An SSIS-based ETL needs to run in Azure post-migration. What are the options?

Answer:

- Lift SSIS into Azure Data Factory's SSIS Integration Runtime.
- Or modernize the pipelines natively in ADF/Synapse.
- Ensure connectivity from the IR to sources/targets.
- Validate package behavior and scheduling after migration.

Q155. You need to move a very large database with minimal transfer time. What helps?

Answer:

- Use backup/restore to Blob with striping, or DMS online sync for large sizes.
- For huge datasets, consider offline seeding via Azure Data Box, then catch up with logs.
- Compress backups and parallelize transfer.
- Test the transfer time against your cutover window.

Q156. How do you validate a migration was successful beyond 'it connects'?

Answer:

- Row counts and checksums on critical tables between source and target.
- Run representative application transactions and key reports.
- Compare performance against a pre-migration baseline (Query Store).
- Confirm jobs, logins, and integrations all function.

Q157. A migration target must match the source SQL version behavior exactly. What do you do?

Answer:

- Choose MI or SQL on VM for high compatibility, and set the matching compatibility level.
- Test the workload at the chosen compatibility level with Query Store.
- Address any behavior changes from the version difference.
- Only raise compatibility level after validation.

Q158. You're modernizing from IaaS SQL to a PaaS tier. How do you choose the tier and size?

Answer:

- Collect the source's CPU/IO/memory profile and map it to vCore sizing.
- Pick General Purpose vs Business Critical based on IO latency and HA needs.
- Consider Hyperscale for very large or fast-growing databases.
- Right-size after go-live using real telemetry, not guesses.

Q159. How do you handle cross-database dependencies when moving to single Azure SQL Database?

Answer:

- Single database doesn't support cross-database queries — refactor or choose MI.
- Consolidate dependent databases onto MI to preserve cross-db queries.
- Or re-architect the app to use separate connections/services.
- Identify all dependencies with DMA before deciding.

Q160. A migration cutover keeps overrunning the window. How do you shrink it?

Answer:

- Move to online/continuous sync so only a small delta remains at cutover.
- Pre-stage as much as possible; keep the final switch minimal.
- Automate the cutover steps to remove manual delay.
- Rehearse to find and eliminate the slow steps.

Q161. You need to migrate SSRS reports alongside the database. What's the path?

Answer:

- Move reports to Power BI Report Server or paginated reports in the Power BI service.
- Re-point data sources to the new Azure database endpoints.
- Validate report rendering and subscriptions.
- Retire the old SSRS instance only after validation.

Q162. Post-migration performance is worse than on-prem. What's your investigation order?

Answer:

- Compare Query Store baselines and check compatibility-level-driven plan changes.
- Confirm the chosen tier/IO profile matches the workload (GP vs BC latency).
- Update statistics and re-check indexing on the target.
- Right-size compute/storage based on the real bottleneck.

Q163. How do you migrate with encryption in place so data is never exposed?

Answer:

- Keep TDE enabled on source and target; migrate over encrypted connections.
- For column-level protection, preserve Always Encrypted keys/config on the target.
- Restrict who can access the migration staging artifacts.
- Validate encryption posture after cutover.

10. Automation, DevOps, IaC & Job Orchestration

DOMAIN 10

Automation, DevOps, IaC & Job Orchestration

14 real-time scenarios

Q164. You need scheduled maintenance (index/stats) across many Azure SQL Databases with no SQL Agent. What do you use?

Answer:

- Use Elastic Jobs to run T-SQL maintenance across a target group of databases on a schedule.
- Or trigger maintenance via Automation runbooks / Azure Functions on a timer.
- Parameterize targets so new databases are picked up automatically.
- Log outcomes centrally and alert on failures.

Q165. Your team wants database schema changes deployed through CI/CD. How do you structure it?

Answer:

- Keep schema in source control (SSDT/DACPAC or migration scripts).
- Build and validate in a pipeline; deploy via SqlPackage/DACPAC to each environment.
- Gate production with approvals and run it as a service principal.
- Capture drift and roll forward with reviewed changes only.

Q166. You must provision identical databases for dev/test/prod repeatably. What's the approach?

Answer:

- Define resources in Bicep/ARM or Terraform and parameterize per environment.
- Store templates in Git and deploy through pipelines.
- Bake in tier, firewall, auditing, and retention so every environment matches.
- Avoid portal click-ops so environments never drift.

Q167. A recurring data refresh from prod to dev must be automated and safe. How do you build it?

Answer:

- Automate copy/restore of prod to a dev target, then run masking/scrubbing.
- Orchestrate with a runbook/pipeline including validation and cleanup.
- Never expose unmasked production data to dev users.
- Schedule it and alert on failures.

Q168. You want to auto-scale a Serverless database's limits based on a schedule. How?

Answer:

- Automate min/max vCore and auto-pause settings via CLI/PowerShell on a timer.
- Or rely on Serverless auto-scaling within the configured range for demand.
- Use runbooks to change bounds ahead of known peaks.
- Monitor to confirm scaling matches the workload.

Q169. How do you manage secrets for automation jobs that touch databases?

Answer:

- Use managed identities for runbooks/functions so no secrets are stored.
- If secrets are unavoidable, keep them in Key Vault and fetch at runtime.
- Grant least-privilege database permissions to the automation identity.
- Rotate and audit credentials.

Q170. You need to enforce that every new database has auditing and backups configured. How?

Answer:

- Use Azure Policy to audit/deny non-compliant databases at scale.
- Apply consistent settings via IaC so compliance is the default.
- Report on policy compliance and remediate exceptions.
- Bake the controls into the provisioning pipeline.

Q171. A PowerShell/dbatools workflow needs to run against Azure targets. What changes vs on-prem?

Answer:

- Authenticate with Entra/managed identity and use Azure-aware connection settings.
- Ensure network access (private endpoint/VNet) from the automation host.
- Some box-SQL cmdlets don't apply to PaaS — use platform APIs where needed.
- Test least-privilege access for the automation principal.

Q172. You want deployments to be zero-touch and auditable. What do you put in place?

Answer:

- Pipeline-driven deployments with approvals, logging, and immutable artifacts.
- Run as a dedicated service principal with scoped permissions.
- Record who approved and deployed each change.
- No manual production changes outside the pipeline.

Q173. How do you orchestrate cross-database jobs that must run in a specific order?

Answer:

- Use Elastic Jobs or a Data Factory/Logic App pipeline to sequence steps with dependencies.
- Add success checks between steps and stop on failure.
- Make steps idempotent so reruns are safe.
- Alert on failure with enough context to act.

Q174. A runbook must scale a database up before a batch and down after. How do you make it robust?

Answer:

- Script the scale-up, wait for completion, run the batch, then scale down.
- Handle the online scaling reconnect and verify the target tier is reached.
- Add error handling and alerting if any step fails.
- Log start/end so cost impact is visible.

Q175. You need to detect and remediate configuration drift across the estate. What's the pattern?

Answer:

- Define desired state in IaC and re-apply/compare regularly.
- Use Azure Policy to flag non-compliant settings continuously.
- Alert on drift and remediate through the pipeline, not by hand.
- Keep a change log for audit.

Q176. How do you automate failover-group failover for planned DR drills?

Answer:

- Script a planned (friendly) failover via CLI/PowerShell with health checks.
- Validate app connectivity and monitoring post-failover in the script.
- Automate failback and capture RTO/RPO for the record.
- Schedule the drill and require sign-off.

Q177. You want a self-service way for developers to get a masked database copy. How?

Answer:

- Provide a pipeline/runbook that copies prod, masks it, and provisions access on request.
- Enforce masking and time-boxed access automatically.
- Track who requested/received copies for audit.
- Clean up copies on a schedule to control cost.

11. Elastic Pools, Serverless & Hyperscale

DOMAIN 11

Elastic Pools, Serverless & Hyperscale

12 real-time scenarios

Q178. You have 200 small databases with spiky, uncorrelated load. What's the cost-efficient design?

Answer:

- Put them in an elastic pool so they share a bucket of resources.
- Uncorrelated spikes mean they rarely peak together, so pooled capacity is cheaper than per-database.
- Size the pool to the aggregate, not the sum of peaks.
- Monitor per-database usage to catch a noisy neighbor.

Q179. One database in an elastic pool is starving the others. How do you handle it?

Answer:

- Set per-database min/max vCore (or DTU) caps within the pool to bound any one database.
- Move a persistently heavy database out to its own resource.
- Right-size the pool if aggregate demand grew.
- Monitor to confirm fairness after the change.

Q180. A database grows past traditional tier limits and needs fast restore. Why Hyperscale?

Answer:

- Hyperscale scales storage to very large sizes with snapshot-based, near-instant backup/restore.
- It decouples compute and storage with a multi-tier caching architecture.
- Read replicas add read-scale without full copies.
- It fits large, fast-growing databases that outgrow other tiers.

Q181. When is Serverless the wrong choice despite its auto-pause savings?

Answer:

- For steady, always-on workloads where provisioned compute is cheaper and avoids warm-up delays.
- Where the first-connection-after-pause latency is unacceptable.
- For latency-critical OLTP that can't tolerate auto-scaling variability.
- Use Serverless for intermittent/unpredictable workloads instead.

Q182. You need read-scale for heavy reporting on a Hyperscale database. What do you use?

Answer:

- Add Hyperscale named/HA replicas and route read-only traffic to them.
- Reads on replicas don't consume primary compute.
- Scale the number of replicas to the read demand.
- Monitor replica lag for reporting consistency expectations.

Q183. An elastic pool's aggregate usage is consistently near the ceiling. What do you do?

Answer:

- Scale the pool up, or split databases across multiple pools by usage profile.
- Identify databases that should live outside the pool.
- Re-balance based on measured per-database consumption.
- Re-evaluate periodically as workloads change.

Q184. How do you move a database into or out of an elastic pool with minimal disruption?

Answer:

- Moving in/out of a pool is an online metadata operation with a brief reconnect.
- Schedule it in a low-traffic window and use retry logic.
- Validate performance after the move.
- Automate for repeatability across many databases.

Q185. A Serverless database's auto-pause is causing cold-start complaints. How do you tune it?

Answer:

- Increase the auto-pause delay so it stays warm during sporadic activity.
- Or disable auto-pause if the workload is frequent enough to negate savings.
- Educate users that the first post-pause connection warms up.
- Balance the pause savings against the latency tolerance.

Q186. You need to forecast elastic pool sizing before onboarding new tenants. How?

Answer:

- Model expected per-tenant load and its correlation with existing tenants.
- Size for the realistic aggregate peak, not the sum of individual peaks.
- Leave headroom and monitor after onboarding.
- Split into multiple pools if profiles diverge.

Q187. Hyperscale compute needs to scale up for a heavy period. What's the behavior?

Answer:

- Scaling compute is fast because storage is decoupled — no full data copy is needed.
- Add/scale replicas or resize compute with minimal disruption.
- Return to the baseline afterward to control cost.
- Automate the scale change around the known window.

Q188. You're deciding between a big single database and an elastic pool for a multi-tenant SaaS. What guides it?

Answer:

- Isolation and per-tenant limits favor separate databases in a pool.
- Simplicity and cross-tenant queries may favor a single database with tenant keys.
- Pools give cost efficiency for many small, spiky tenants.
- Choose based on isolation, cost, and query patterns.

Q189. A Hyperscale database needs DR. What options apply and what's the advantage?

Answer:

- Use geo-replication/failover groups; Hyperscale's architecture enables fast seeding.
- Failover is quick relative to restoring large traditional databases.
- Confirm the specific replica options in your region/configuration.
- Test failover to validate behavior and timing.

12. Networking & Connectivity

DOMAIN 12

Networking & Connectivity

6 real-time scenarios

Q190. Your app in a VNet must reach Azure SQL Database without going over the public internet. How?

Answer:

- Create a private endpoint for the SQL server and disable public network access.
- Traffic stays on the Microsoft backbone/VNet via a private IP.
- Ensure private DNS resolves the server FQDN to the private endpoint.
- Validate that public connectivity is refused.

Q191. Clients connect but experience extra latency; you suspect the connection policy. What do you check?

Answer:

- Check Proxy vs Redirect connection policy — Redirect lowers latency after the initial gateway handshake.
- Use Redirect for in-VNet clients where possible for direct connections.
- Confirm required ports for Redirect are open through NSGs/firewalls.
- Measure latency before and after the policy change.

Q192. A Managed Instance can't resolve an on-prem host over VPN. What's the likely issue?

Answer:

- DNS resolution from the MI subnet isn't reaching on-prem DNS.
- Configure custom DNS on the VNet or conditional forwarding to on-prem.
- Confirm the VPN/ExpressRoute path and route tables are correct.
- Test name resolution from within the subnet.

Q193. You must restrict which subnets can talk to the database. What layers do you use?

Answer:

- Use private endpoints plus NSGs to permit only the required source subnets.
- Apply VNet rules/firewall rules scoped to those ranges.
- Disable broad public access.
- Verify with a connectivity test from allowed and disallowed sources.

Q194. Cross-region traffic between app and database is costly and slow. How do you improve it?

Answer:

- Co-locate the app and database in the same region to cut latency and egress.
- Use read replicas in the app's region for read-heavy workloads.
- Keep writes near the primary and design for regional locality.
- Measure the latency/cost improvement.

Q195. An outbound firewall blocks the SQL endpoint after a network change. How do you diagnose connectivity fast?

Answer:

- Test the port/endpoint reachability from the client host and check NSG/route/firewall logs.
- Confirm private DNS resolves to the expected private IP.
- Check for recent NSG or route-table changes.
- Restore the required rule and validate the connection.

13. Cost Optimization, Licensing & Governance

DOMAIN 13

Cost Optimization, Licensing & Governance

5 real-time scenarios

Q196. You want to cut steady-state Azure SQL compute cost without hurting performance. What levers apply?

Answer:

- Buy reserved capacity for predictable, always-on databases.
- Apply Azure Hybrid Benefit to reuse existing licenses with Software Assurance.
- Right-size tiers using real telemetry and remove unused replicas/features.
- Use elastic pools for many small databases.

Q197. How do you allocate database costs back to teams accurately?

Answer:

- Tag resources by owner/app/environment and use Cost Management for allocation.
- Group into resource groups aligned to ownership.
- Report spend per tag and review with owners.
- Set budgets and alerts per team.

Q198. A large estate has drifted into over-provisioned tiers. How do you right-size systematically?

Answer:

- Collect utilization (CPU/IO/memory) per database and compare to tier caps.
- Downshift over-provisioned databases and consolidate into pools where sensible.
- Validate performance after each change with Query Store.
- Make right-sizing a recurring review, not a one-off.

Q199. Management wants a forecast of next quarter's database spend. How do you produce it?

Answer:

- Trend historical spend and factor in known growth, new projects, and reservations.
- Model tier/scaling changes and reservation savings.
- Present the forecast with assumptions and confidence.
- Track actuals against the forecast monthly.

Q200. You must enforce governance so nobody provisions expensive tiers without approval. How?

Answer:

- Use Azure Policy to restrict allowed SKUs/tiers and require tags.
- Deny or audit non-compliant provisioning at the subscription/management-group scope.
- Route exceptions through an approval process.
- Report on compliance and cost impact.

End of Document

Compiled by Praveen Madupu (PMSQLDBA) - sqldbachamps.com. 200 advanced real-time Azure DBA scenarios across 13 domains (Volume 2).